COSTAR: IMPROVED TEMPORAL COUNTERFACTUAL ESTIMATION WITH SELF-SUPERVISED LEARNING

Week 6: July 4^{th} to July 11^{th}

Background

- Temporal counterfactual outcome estimation is crucial for decision-making in domains like healthcare and e-commerce
- In the real world, hard to conduct randomized controlled trials (RCTs) due to cost or ethical/practical constraints
- Instead, rely on **observational time-series data**, where past covariates and treatments influence future outcomes

Problem

- **Modeling time-dependent confounding** is difficult due to:
    - Complex temporal dynamics
    - Long-range dependencies
    - Interplay between covariates, treatments, and outcomes
- Existing methods (like RMSN, CRN, G-Net, Causal Transformer) use RNNs or Transformers but:
    - Rely on **fully supervised training** using future outcome labels
    - Struggle to generalize to **unseen subpopulations** or **cold-start settings**
    - Can't leverage the abundance of unlabeled data effectively; **Existing time-series self supervised learning (SSL) methods aren't tailored for counterfactual outcome estimation**, and **counterfactual works haven't explored SSL**

Contribution

- **COSTAR**: Proposes a **Counterfactual Self-Supervised Transformer** that:
    - Integrates **SSL** into temporal counterfactual estimation
    - Learns expressive and **transferable representations** of history
- **Component-wise Contrastive Learning**:
    - Refines SSL by introducing **separate contrastive losses** for covariates, treatments, and outcomes: improving robustness and interpretability
- **Theoretical Analysis**:
    - Provides a **generalization error bound** from an **unsupervised domain adaptation** perspective: explains why SSL helps in counterfactual prediction

Problem setting

- Dataset: $\{\{\boldsymbol{x}_t^{(i)}, \boldsymbol{a}_t^{(i)}, \boldsymbol{y}_t^{(i)}\}_{t=1}^{T^{(i)}}, \boldsymbol{v}^{(i)}\}_{i=1}^{N}$ of N independently sampled subjects,
  - Each sample (subject) has:
  - A time series of: Covariates x_t∈R^{d_X}; Treatments a_t∈R^{d_A}; Outcomes y_t∈R^{d_Y}; Static features v∈R^{d_V}
- Objective:
  - At any time t, given observed history $\bar{H}_t = (x_{1:t}, a_{1:t-1}, y_{1:t}, v)$ and a proposed treatment sequence a_{t:t+τ−1}, estimate:

$$\mathbb{E}[y_{t+\tau}[a_{t:t+\tau-1}] \mid \bar{H}_t]$$

  Distribution shift: source/target domain:

$$P_{\mathcal{D}_{tr}}(h) \text{ and } P_{\mathcal{D}}(h)$$

- Assumption:
  - **Consistency:** If a treatment a_t is actually given, then the observed outcome equals the counterfactual:

$$y_{t+1}[a_t] = y_{t+1}$$

  - **Positivity:** Every possible treatment has positive probability of being observed, given past history:

$$P(a_t = a \mid \bar{A}_{t-1}, \bar{X}_t) > 0$$

  - **Sequential Strong Ignorability:** No unmeasured confounding at each time step, conditional on past history:

$$\boldsymbol{Y}_{t+1}[\boldsymbol{a}_t] \perp\!\!\!\perp \boldsymbol{A}_t | \bar{\boldsymbol{A}}_{t-1}, \bar{\boldsymbol{X}}_t, \forall \boldsymbol{a}_t, t.$$

Counterfactual Self-Supervised Transformer

COSTAR is designed to accurately estimate **counterfactual treatment outcomes over time** by learning expressive and transferable representations from observational data.

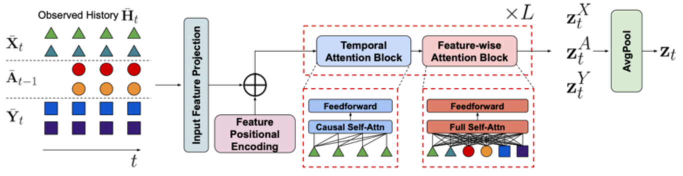
- Main goal: To create a unified model that learns from observed history, generalizes well to unseen treatment plans or subpopulations, and enables efficient counterfactual reasoning.
- Core components:
  - Representation Learning; Self-Supervised Loss Design; Non-Autoregressive Decoder

Counterfactual Self-Supervised Transformer

- **Encoder architecture**
  - **Goal:** Design a Transformer-based encoder that effectively learns representations from **observed temporal history** H̄t — including covariates, treatments, and outcomes — for accurate and generalizable **counterfactual outcome estimation**.
  - Input Structure

$$\bar{H}_t = (\bar{X}_t, \bar{A}_{t-1}, \bar{Y}_t) \in \mathbb{R}^{t \times d_{\text{input}}}, \quad \text{where } d_{\text{input}} = d_X + d_A + d_Y$$

  - Architecture Design
    - **Feature-wise Attention Block**: Applies **self-attention across features** (at each time step) to model inter-feature interactions; Includes both **time-varying** and **static** features



(a) Encoder architecture.

$$\boldsymbol{E}^S[i,j] = f_{\text{input}}(\bar{\boldsymbol{S}}_t[i,j]),\ 1 \leq i \leq t,\ 1 \leq j \leq d_S;$$
$$\boldsymbol{E}^V[j] = f_{\text{input}}(\boldsymbol{V}[j]),\ 1 \leq j \leq d_V.$$

      - Feature positional encoding: informs group identity & position within group
        - helping the Transformer "know" which feature is which in time series data
        - Transformers, by default, treat sequences as unordered, unless we inject structure like positional encodings
        - we also need to distinguish which feature slot is which — since different columns (say "age" vs "blood pressure") have different semantics, even if their values are numerically close

$$\boldsymbol{E}_{\text{fea\_pos}}(f_i^{\boldsymbol{F}}) = \boldsymbol{E}_{\text{fea}} \cdot \text{Concat}(\boldsymbol{e_F}, \boldsymbol{e}_i), \text{ where}$$
$$\boldsymbol{e_F} = \begin{cases} (1,0,0,0) \text{ if } \boldsymbol{F} \text{ is } \boldsymbol{F}_X \\ (0,1,0,0) \text{ if } \boldsymbol{F} \text{ is } \boldsymbol{F}_A \\ (0,0,1,0) \text{ if } \boldsymbol{F} \text{ is } \boldsymbol{F}_Y \\ (0,0,0,1) \text{ if } \boldsymbol{F} \text{ is } \boldsymbol{F}_V \end{cases},$$

$e_F \in \mathbb{R}^4$ is a **one-hot vector** that says: is this feature from **X, A, Y, or V**?

$e_i \in \mathbb{R}^{\max(d_X, d_A, d_Y, d_V)}$ is a **one-hot vector** indicating this is the **i-th feature** inside group F

$E^{\text{fea}} \in \mathbb{R}^{d_{\text{model}} \times (\cdots)}$ is a **learnable weight matrix**, shared across all features.

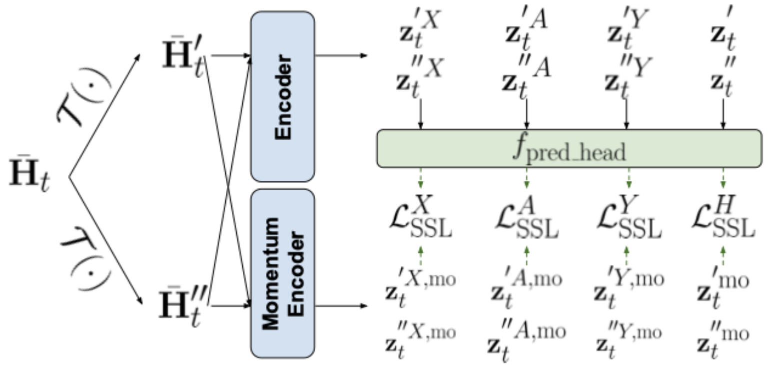
    - **Temporal Attention Block**:
      - Applies **causal self-attention (future masked) along the time dimension**, independently for each feature.
      - Ensures representations only access current and past time steps (no future leakage).
      - aware of past → future structure through positional relation of layers
    - **Average pooling:** gives a **summary vector** — like a centroid — capturing what the covariate "block" collectively expresses

$$\boldsymbol{z}_t^X = \text{avg}(\{\boldsymbol{z}_t^i\}_{f_i \in \mathbf{F}_X}), \quad \boldsymbol{z}_t^A = \text{avg}(\{\boldsymbol{z}_t^i\}_{f_i \in \mathbf{F}_A}),$$
$$\boldsymbol{z}_t^Y = \text{avg}(\{\boldsymbol{z}_t^i\}_{f_i \in \mathbf{F}_Y}), \quad \boldsymbol{z}_t = \text{avg}(\{\boldsymbol{z}_t^i\}_{i=1}^{d_{\text{input}}}).$$

$$\underbrace{d_X \times d_{\text{model}}}_{\text{Before pooling}} \xrightarrow{\text{avg over rows}} \underbrace{1 \times d_{\text{model}}}_{\text{After pooling}}$$

Counterfactual Self-Supervised Transformer

- **Self-supervised representation learning of the observed history**
  - Goal: To learn **informative, transferable, and balanced representations** of the observed history H¯t using **self-supervised contrastive learning**, instead of relying solely on supervised outcome labels.
  - **Contrastive Learning Framework**: learn **balanced** representations; useful for cold-start
    - Each input history H¯t is augmented twice using random transformations (scaling, shifting, jittering) to create **positive pairs; And negative pairs**:
      Other samples from the same batch (different patients/items)
    - training the encoder to **consistently represent the same subject across augmentations**, while **keeping it distinguishable from others**



(b) Self-supervised learning of history.

two views per input, i.e. **positive pairs** (H¯t′(i),H¯t′′(i))

$$
\begin{aligned}
\boldsymbol{z}_t^{\prime X(i)}, \boldsymbol{z}_t^{\prime A(i)}, \boldsymbol{z}_t^{\prime Y(i)}, \boldsymbol{z}_t^{\prime(i)} &= f_{\text{enc}}(\bar{\boldsymbol{H}}_t^{\prime(i)}),\\
\boldsymbol{z}_t^{\prime\prime X(i)}, \boldsymbol{z}_t^{\prime\prime A(i)}, \boldsymbol{z}_t^{\prime\prime Y(i)}, \boldsymbol{z}_t^{\prime\prime(i)} &= f_{\text{enc}}(\bar{\boldsymbol{H}}_t^{\prime\prime(i)}),\\
\boldsymbol{z}_t^{\prime X,\text{mo}(i)}, \boldsymbol{z}_t^{\prime A,\text{mo}(i)}, \boldsymbol{z}_t^{\prime Y,\text{mo}(i)}, \boldsymbol{z}_t^{\prime \text{mo}(i)} &= f_{\text{enc}}^{\text{mo}}(\bar{\boldsymbol{H}}_t^{\prime(i)}),\\
\boldsymbol{z}_t^{\prime\prime X,\text{mo}(i)}, \boldsymbol{z}_t^{\prime\prime A,\text{mo}(i)}, \boldsymbol{z}_t^{\prime\prime Y,\text{mo}(i)}, \boldsymbol{z}_t^{\prime\prime \text{mo}(i)} &= f_{\text{enc}}^{\text{mo}}(\bar{\boldsymbol{H}}_t^{\prime\prime(i)}).
\end{aligned}
$$

$$\theta_{\text{mo}} \leftarrow m \cdot \theta_{\text{mo}} + (1-m)\cdot\theta$$

Momentum encoder (Used in MoCo v3 and adopted by COSTAR): produce **stable target embeddings** for contrastive learning

- Problem in contrastive learning without it:
  - Query and key are both being updated every step → they **chase each other**
  - The positive pair becomes noisy → unstable learning
- Solution:
  - Query comes from the **main encoder** (updated every step)
  - Key comes from the **momentum encoder** (stable, smoother)

$$
\begin{aligned}
\mathcal{L}_{\text{SSL}}^{H} =& \mathcal{L}_{\text{InfoNCE}}(\{\boxed{f_{\text{pred\_head}}}(\boldsymbol{z}_t^{\prime(i)})\}_{i=1}^{B}, \{\boldsymbol{z}_t^{\prime\prime\text{mo}(i)}\}_{i=1}^{B})\\
&+ \mathcal{L}_{\text{InfoNCE}}(\{f_{\text{pred\_head}}(\boldsymbol{z}_t^{\prime\prime(i)})\}_{i=1}^{B}, \{\boldsymbol{z}_t^{\prime\text{mo}(i)}\}_{i=1}^{B}),
\end{aligned}
$$

$f_{\text{pred head}}(z)$: the anchor (query) in contrastive space

$z''^{,\text{mo}}$: the key (target) in contrastive space

$$
\begin{aligned}
L_{\text{SSL}}^{H} =& \underbrace{L_{\text{InfoNCE}}(\{f_{\text{pred}}(z_t^{\prime(i)})\}_{i=1}^{B}, \{z_t^{\prime\prime(i),\text{mo}}\}_{i=1}^{B})}_{\text{pull + push: view 1} \leftrightarrow \text{view 2 (momentum)}}\\
&+\underbrace{L_{\text{InfoNCE}}(\{f_{\text{pred}}(z_t^{\prime\prime(i)})\}_{i=1}^{B}, \{z_t^{\prime(i),\text{mo}}\}_{i=1}^{B})}_{\text{pull + push: view 2} \leftrightarrow \text{view 1 (momentum)}}
\end{aligned}
$$

Role of f_{pred_head}:

$$q = f_{\text{pred}}(z_t) \in \mathbb{R}^{d_{\text{proj}}}$$

- MLP **transforms** the encoder's output z_t∈R^{d_model} into a new space used **only** for computing the **contrastive loss**.
- Allows the encoder to focus on general-purpose representations,
- While the projection head can focus on contrastive geometry (pulling/pushing).

$$
\mathcal{L}_{\text{InfoNCE}}\left(\boxed{\{\boldsymbol{q}^{(i)}\}_{i=1}^{B}, \{\boldsymbol{k}^{(i)}\}_{i=1}^{B}}\right) = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(\cos\langle\boldsymbol{q}^{(i)}, \boldsymbol{k}^{(i)}\rangle)}{\sum_{j=1}^{B}\exp(\cos\langle\boldsymbol{q}^{(i)}, \boldsymbol{k}^{(j)}\rangle)}.
$$

For each pair:
Pull the **matching pair** (numerator) close
Push away all **non-matching pairs** (denominator)

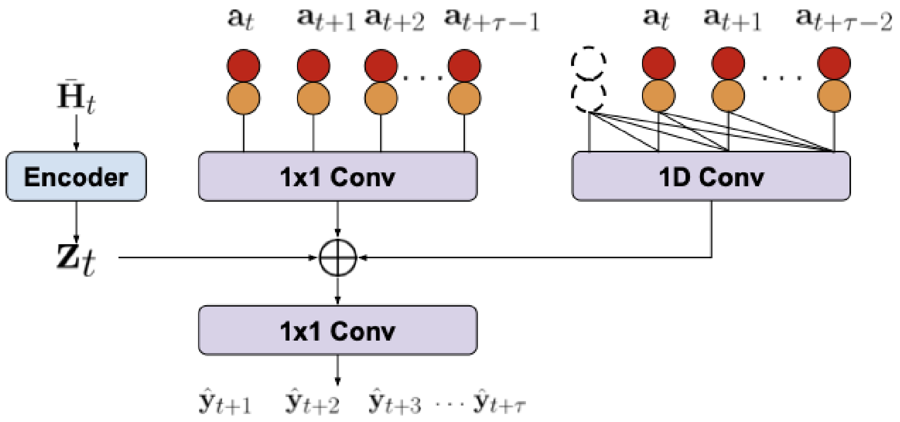
Total loss function:

$$L_{\text{SSL}} = L_{\text{SSL}}^{H} + \frac{1}{3}(L_{\text{SSL}}^{X} + L_{\text{SSL}}^{A} + L_{\text{SSL}}^{Y})$$

Counterfactual Self-Supervised Transformer

- **Unsupervised domain adaptation view of counterfactual outcome estimation:**
  - Predict future outcomes y^{t+1},⋯,y^{t+τ} **in parallel**, without relying on previous predicted outcomes — improving both **efficiency** and **robustness**.
  - **Some intuition:** abductive reasoning: Even if we don't feed previous predicted outcomes explicitly (the auto-regressive mode with update history one by one), the **structure** of the treatment plan might reflect how the world evolved — and thus, helps the model predict what outcomes will happen under that plan
  - the model **assumes** that the future treatment plan is **reasonable** or **plausible**, not arbitrary noise
  - **Components:**
    - **1×1 convolution**: *Per-step treatment encoder*
      - Input: treatment at a **specific future time step** a_t′∈R^{d_A}
      - Role:
        - A **1×1 conv** applies a learned linear transformation independently to each time step: $a_{t'} \in \mathbb{R}^{d_A} \rightarrow h_{t'} \in \mathbb{R}^{h}$
        - No time information involved — this is just **embedding each treatment step**
    - **1D convolution**: *Temporal treatment sequence encoder*
      - Input: the **entire treatment sequence:** $\bar{a}_{t:t+\tau-1} \in \mathbb{R}^{\tau \times d_A}$
      - Role:
        - Slides a filter (kernel) across **time**
        - Learns **patterns across adjacent treatments;**
        - detecting whether treatments are **rising**, **falling**, **cyclic**, etc.
    - **MLP (Multi-Layer Perceptron)**: *Outcome predictor*
      - Concat(encoded history,encoded treatments)
    - **Loss function:**

$$\mathcal{L}_{\text{est}} = \sum_{i=1}^{\tau} w_i \left\|\hat{\boldsymbol{y}}_{t+i} - \boldsymbol{y}_{t+i}\right\|_2^2,$$



(c) Non-autoregressive outcome predictor.

Counterfactual Self-Supervised Transformer

- **Unsupervised domain adaptation view of counterfactual outcome estimation**
  - **COSTAR setting: what they discussed of counterfactual questions are ‘change future treatments’; instead of retrospective**
    - “Given history up to time t, what if I apply **a new treatment plan starting at t** — how will outcomes evolve?”
  - Unsupervised Domain Adaptation (UDA) framing
    - frames counterfactual estimation as a **UDA problem:**
      - S_a: samples where **treatment = a**, so we can learn to predict outcomes for that treatment → this is our **source domain (labeled)**
      - T_a: samples where **treatment ≠ a**, so we don’t observe y[a] → this is our **target domain (unlabeled)**
    - Target: training a model to estimate:

$$f_a(h) \approx \mathbb{E}[Y[a] \mid H = h]$$

    you can only **supervise** it using samples in S_a, and try to **generalize** to samples in T_a. (treatment as independent variable)
    - Domain shift: $P(h \mid a) \neq P(h \mid a')$
    - Generalizability issue: the model trained on S_a (labeled) may **not generalize** well to T_a (unlabeled) due to treatment assignment bias
    - Upper bound of counterfactual outcome estimator:

$$\mathcal{E}_{\mathcal{D}}(f_{\boldsymbol{a}}) \lesssim P(\boldsymbol{a})\mathcal{E}_{\mathcal{S}_{\boldsymbol{a}}}(f_{\boldsymbol{a}}) + (1 - P(\boldsymbol{a})) \cdot \left[\epsilon^2 + (4B^2 - \epsilon^2)\frac{r}{\alpha^2\gamma^4} \cdot \exp(-\Omega(\frac{\rho\gamma^2}{\alpha^2}))\right],$$

    - General idea of proof:
      - Goal: bound the **expected error** of a counterfactual predictor f_a(h): $\mathbb{E}_D[\ell(f_a(h), y[a])]$
      - Decomposition of generalization error: The **first term** is factual — we can compute and optimize it during training. The **second term** is counterfactual — we **can't directly supervise** it.

$$\mathbb{E}_D[\ell(f_a(h), y[a])] = P(a) \cdot \mathbb{E}_{\mathcal{S}_a}[\ell(f_a(h), y)] + (1 - P(a)) \cdot \mathbb{E}_{\mathcal{T}_a}[\ell(f_a(h), y[a])]$$

      - => “How can we **bound** the second term (target domain error) using tools from contrastive learning?”

Counterfactual Self-Supervised Transformer

- **Unsupervised domain adaptation view of counterfactual outcome estimation**
  - Main Steps:
    - Use a representation space learned via contrastive loss
      - Let r(h)∈R^k be the **representation** of history h learned via **contrastive learning;** assume r minimizes a **generalized spectral contrastive loss** (from HaoChen et al., 2022), which clusters similar representations and enforces structure
    - Use graph expansion to relate domains
      - construct a **positive pair graph** over the representation space. Each node is a sample h, and edges connect samples that form positive pairs in contrastive learning; The **graph structure** reflects the quality of the learned representation space
      - **Three key assumptions** about the structure of this graph:
        - **Cross-cluster connections are small** → representations for different subgroups don't overlap too much
          - Representations from S_a and T_a are **distinct but not noisy**
          - No confusion across very different groups
        - **Intra-cluster conductance is large** → positive pairs are tightly clustered
          - The learned representation for each sample is **stable**
          - Contrastive learning brings **same-sample views close**, forming compact clusters
        - **Relative expansion between clusters is high** → domains are not isolated;
          - The representations of counterfactual cases (unlabeled) **lie in similar regions** to the factual ones (labeled);
          - The predictor trained on S_a can **reliably interpolate or extrapolate** to T_a
    - apply the UDA theory from HaoChen et al. (2022)

$$\mathbb{E}_{\mathcal{T}_a}[\ell(f_a(h), y[a])] \leq \epsilon^2 + (4B^2 - \epsilon^2) \cdot E_{01}^{\mathcal{T}_a}(f)$$

    - E_{01} : classification error of a **nearest-cluster-style classifier** in representation space

Takeaway:

- COSTAR framework: predict **future counterfactual outcomes** using **observational time-series data**, even under **distribution shifts** (e.g., cold-start or unseen subpopulations), without relying entirely on labeled outcomes
- SSL: learn **robust, disentangled, and transferable representations** from unlabeled data
- Non-autoregressive: **convolution + MLP** to predict the entire future outcome trajectory in **parallel**
- **generalization bound** from an **unsupervised domain adaptation (UDA)** perspective

Discussion:

- Concern regrading non-autoregressive: treatment sequence in future not given or just nonsense/not reasonable?
- Interventional instead of counterfactual: no need for iteratively unrolling for changing treatment at time t or later, but if past treatment changes?